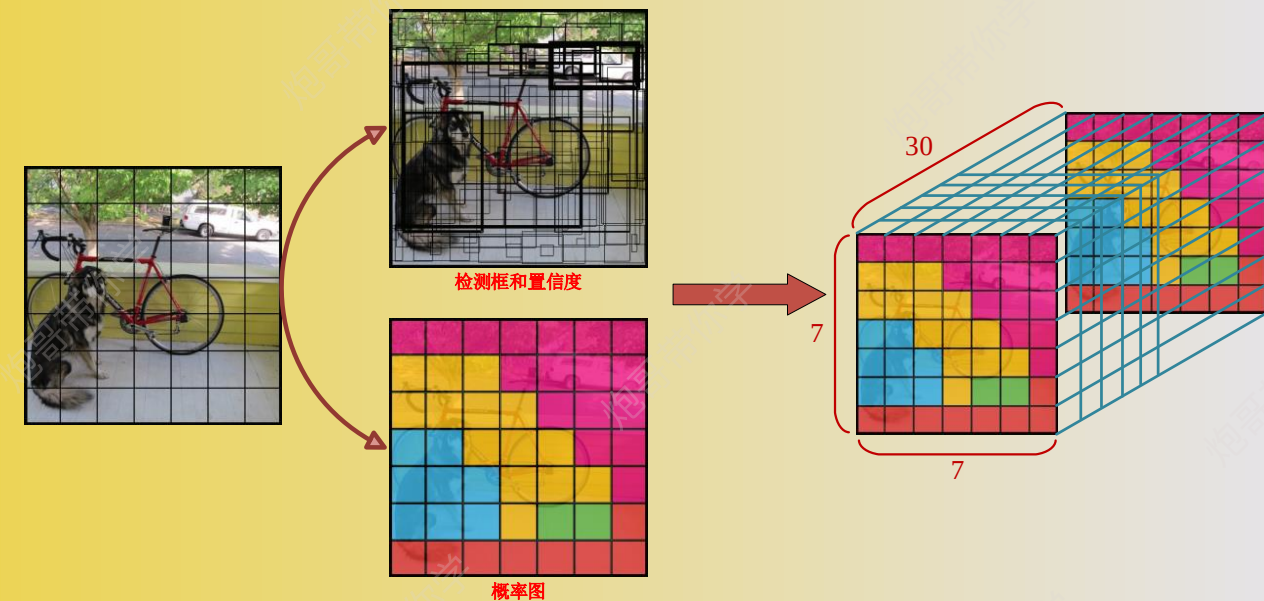




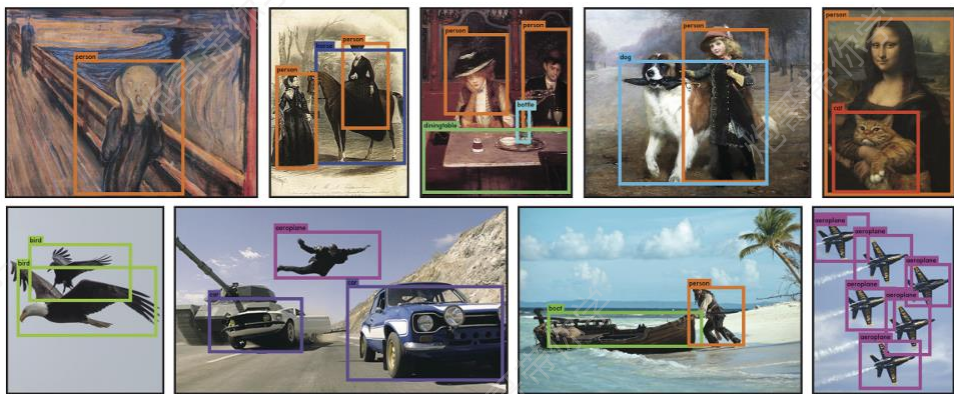
第2章

Yolov1

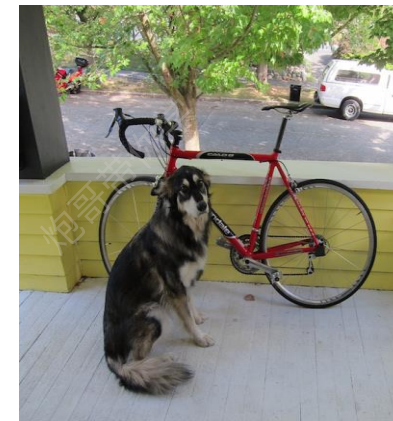
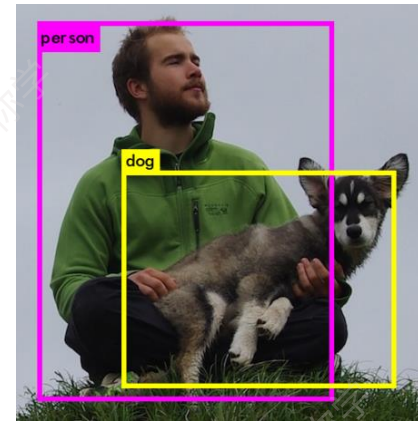
目标检测

算法原理

Yolo目标检测的背景

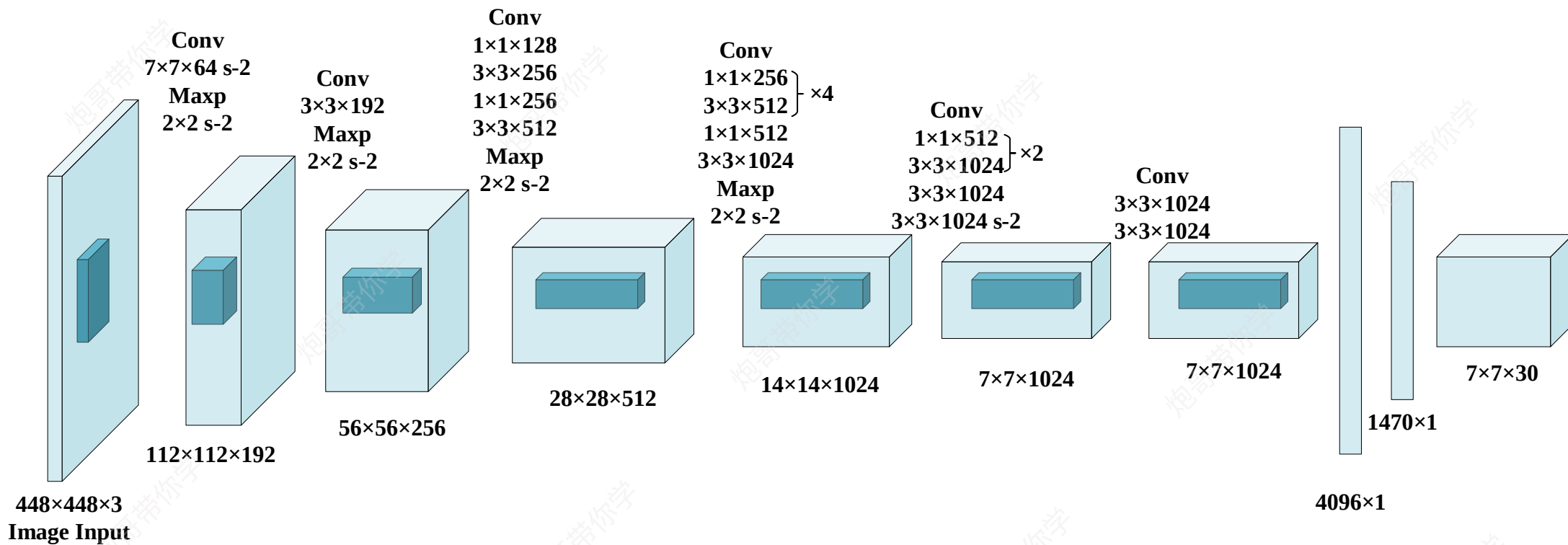


目标检测的两个步骤可以概括为：
步骤一：检测目标位置（生成矩形框）
步骤二：对目标物体进行分类
主流的算法分为两种
单阶段的，例如yolo系列算法
双阶段，例如faster RCNN



YOLO（You Only Look Once）是一种目标检测算法，由 Joseph Redmon等人在2015年提出。它的主要思想是将目标检测任务看作是一个回归问题，并且可以在一个神经网络中同时预测目标的位置和类别。该算法被命名为YOLOv1。其中在2016年，YOLOv2发布，YOLOv3在2018年发布

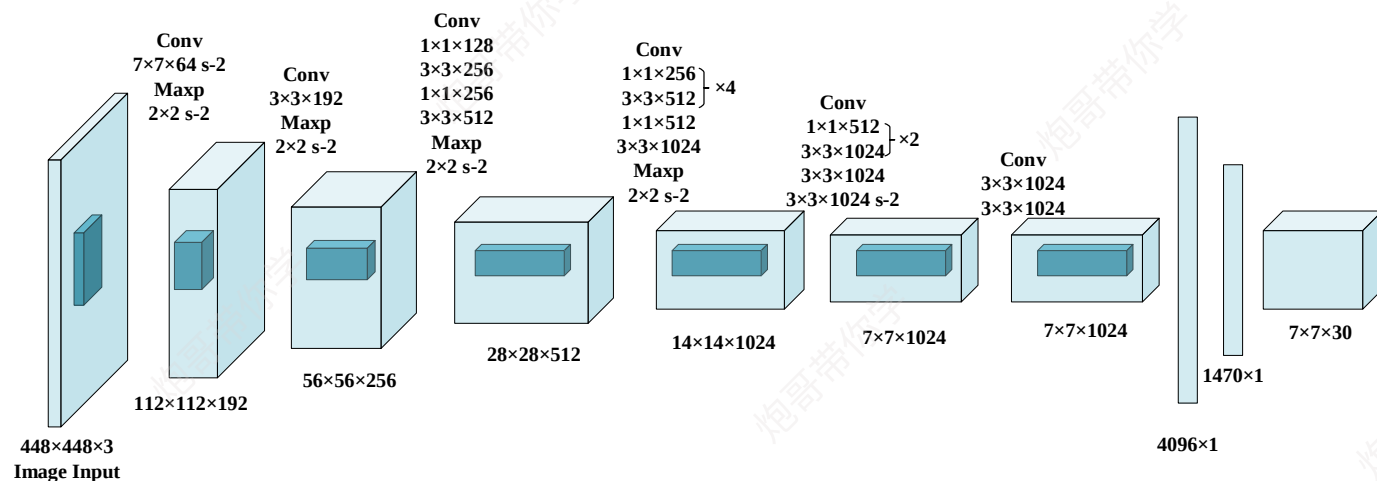
Yolov1网络结构



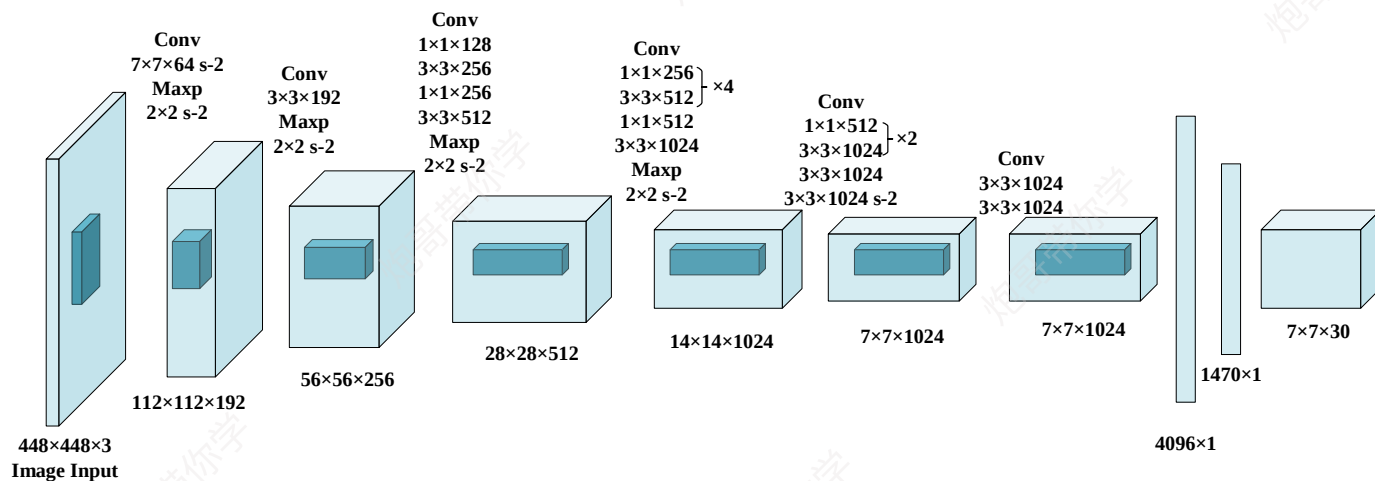
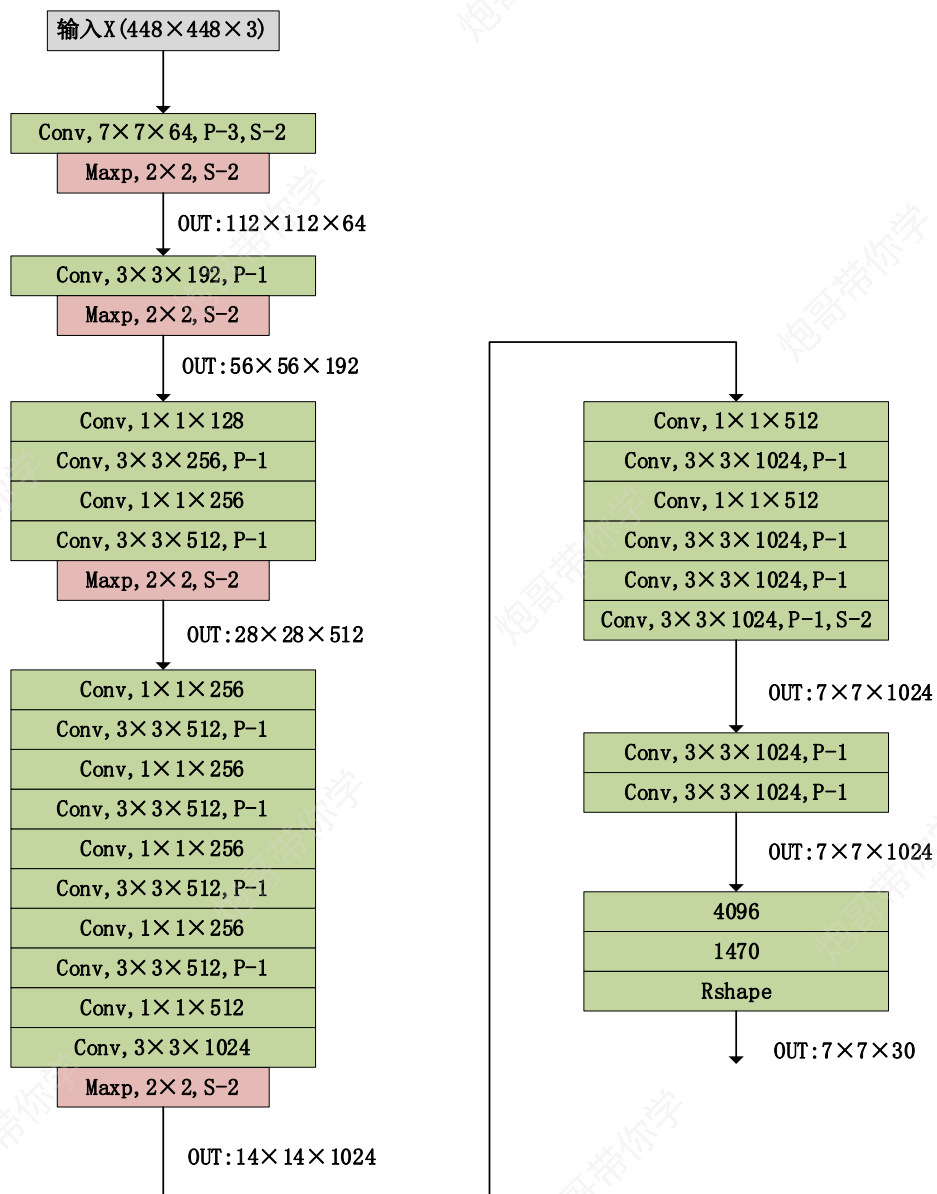
Yolov1网络结构

1、输入图像大小为 448×448 ，经过若干个卷积层与池化层，变为 $7 \times 7 \times 1024$ 张量。

2、最后经过两层全连接层，输出张量维度为 $7 \times 7 \times 30$ ，这就是Yolov1的整个神经网络结构，和一般的卷积分类网络没有太多区别。



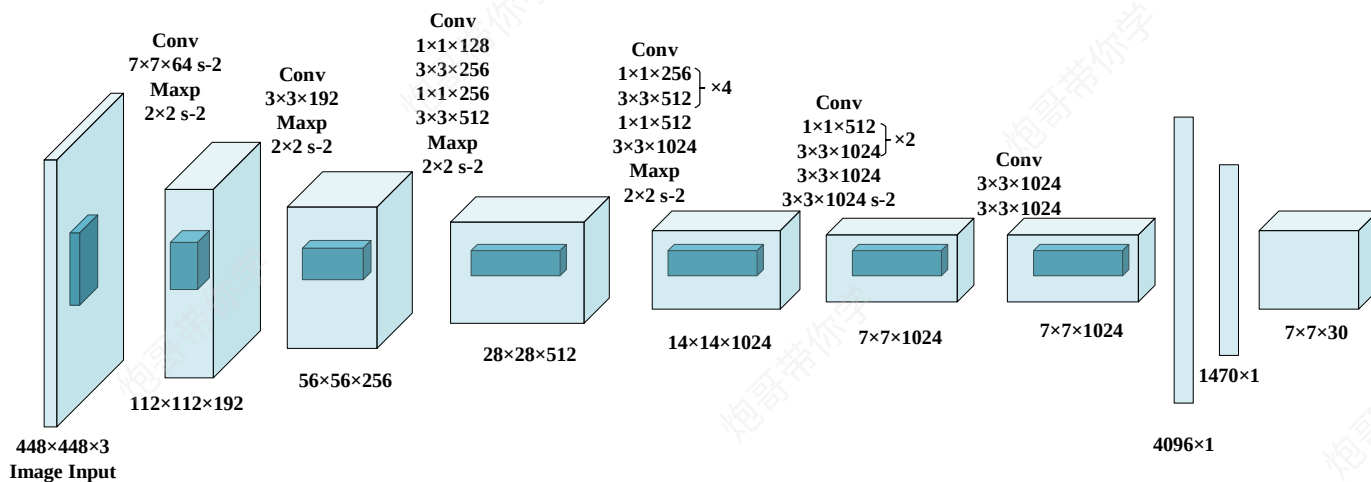
Yolov1网络参数



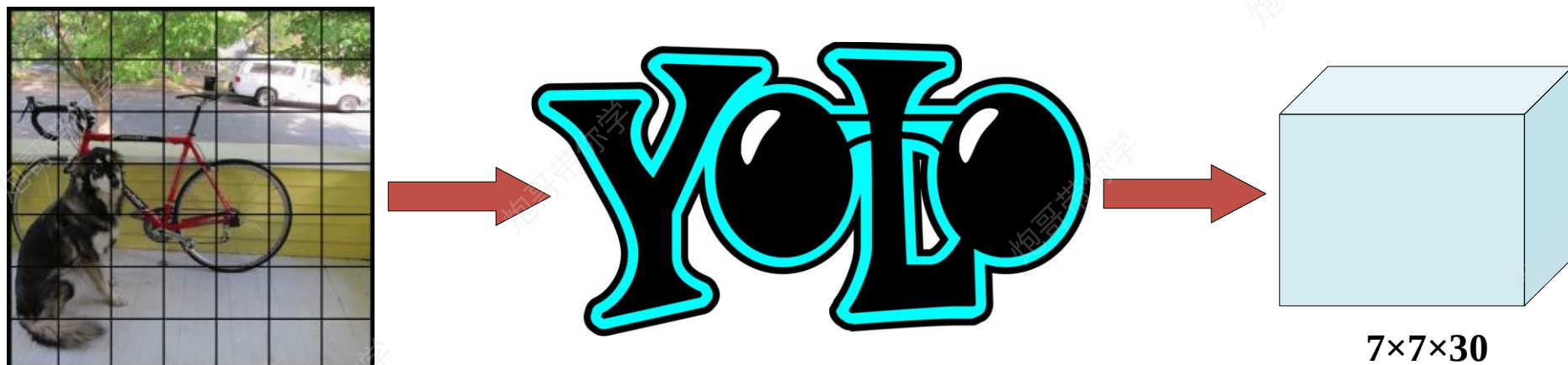
- 1、除了最后一层使用了线性激活函数外，其余层的激活函数为 Leaky ReLU。
- 2、在训练中使用了 Dropout 与数据增强的方法来防止过拟合。
- 3、对于最后一个卷积层，它输出一个形状为 $(7, 7, 1024)$ 的张量。然后张量展开。使用2个全连接层作为一种线性回归的形式，它输出1470个参数，然后reshape为 $(7, 7, 30)$ 。

Yolov1网络参数

Input: 448×448×3				
	Filter	pading	stride	Output
Conv	7×7×64	3	2	224×224×64
MaxPool	2×2		2	112×112×64
Conv	3×3×192	1		112×112×192
MaxPool	2×2		2	56×56×192
Conv	1×1×128			56×56×128
	3×3×256	1		56×56×256
	1×1×256			56×56×256
	3×3×512	1		56×56×512
MaxPool	2×2		2	28×28×512
Conv	1×1×256			28×28×256
	3×3×512	1		28×28×512
	1×1×256			28×28×256
	3×3×512	1		28×28×512
	1×1×256			28×28×256
	3×3×512	1		28×28×512
	1×1×256			28×28×256
	3×3×512	1		28×28×512
	1×1×512			28×28×512
	3×3×1024			28×28×1024
MaxPool	2×2		2	14×14×1024
Conv	1×1×512			14×14×512
	3×3×1024	1		14×14×1024
	1×1×512			14×14×512
	3×3×1024	1		14×14×1024
	3×3×1024	1		14×14×1024
	3×3×1024	1	2	7×7×1024
Conv	3×3×1024	1		7×7×1024
	3×3×1024	1		7×7×1024



Yolov1模型输出结果解析

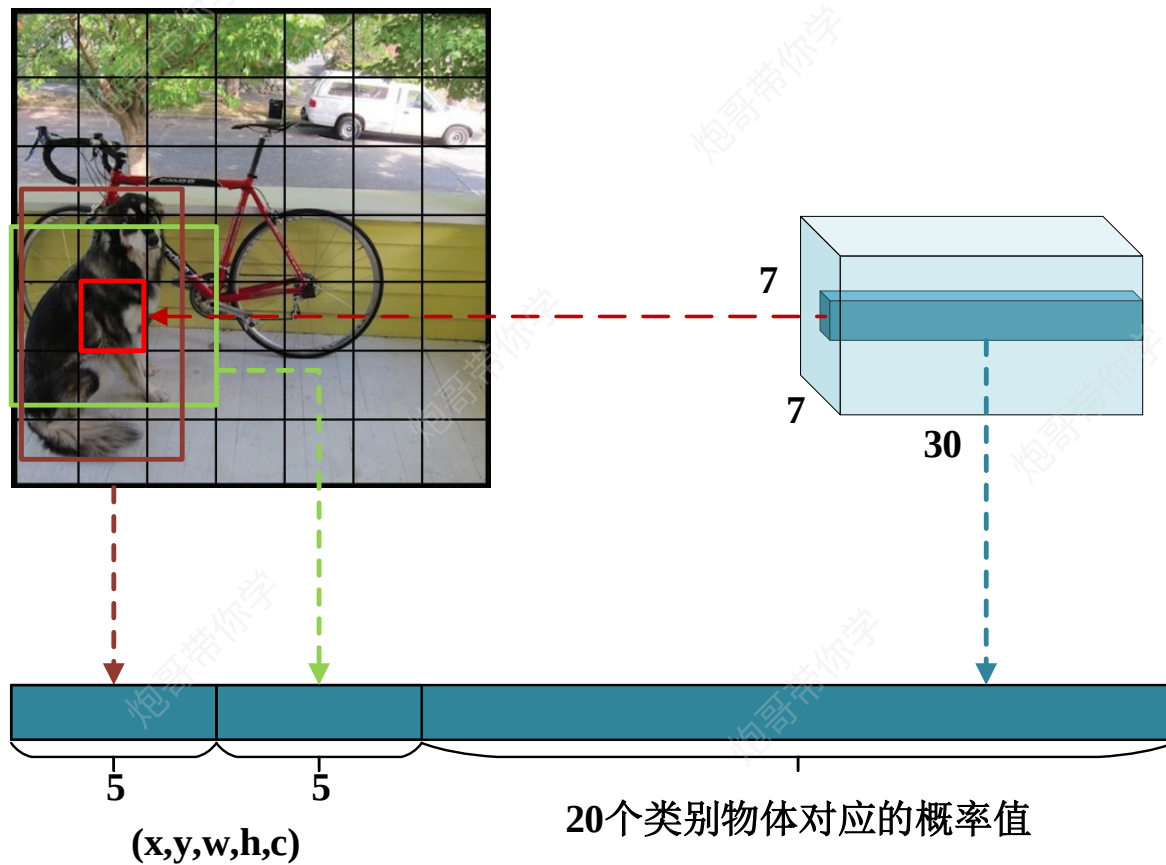


通过前面的分析，目标检测模型和图像分类模型并没有太大的区别，都是输入图像，经过模型，输出结果向量。总结如下：

- 1、输入448×448大小的图片（后续版本这个大小会变化）
- 2、经过YOLOv1模型前向传播计算
- 3、输出7×7×30大小向量，对应原始图片对应区域的结果。就是所谓的将原始图片划分为7×7个小方格（grid cell）大小的图片。

这里需要注意的是将原始图片划分为7×7个小方格并不是真正将图片划分为7×7个小方格，图片还是448×448大小输入网络的，只不过是图片输出的结果向量是7×7，每个结果向量都对应图片的对应区域。

Yolov1模型输出结果解析



对于每个grid cell:

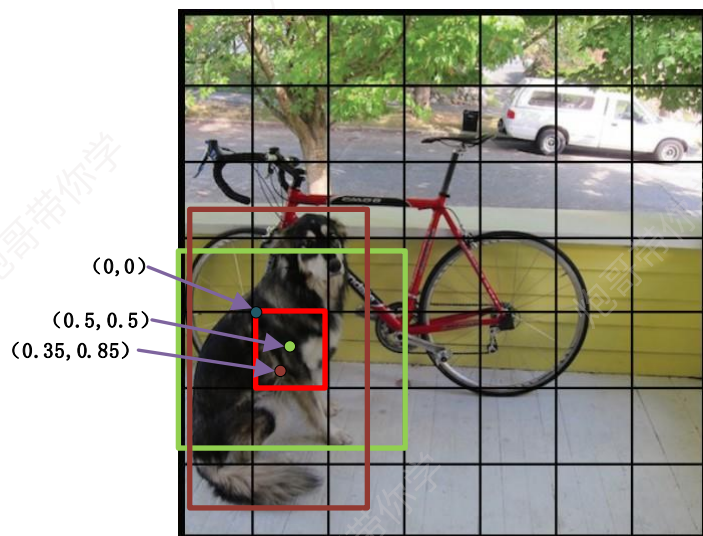
(1) 预测B个边界框 (在yolov1中选择2个边界框作为预测结果)。

(2) 每个边界框包含5个元素, 分别为, 中心坐标: x, y ; 检测框的宽和高: w, h ; 对应检测框的置信度 c

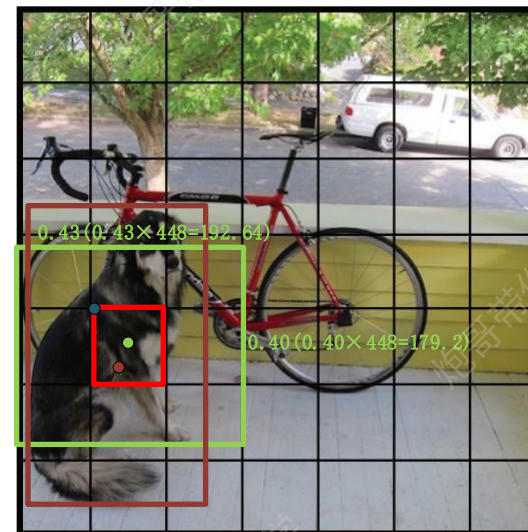
(3) 对应该grid cell (**注意, 不是边界框**) 预测物体的概率值 (利用softmax输出概率)

(4) 综上所述, 因此每个grid cell输出的数据维度为 30×1 , 而 7×7 个方格输出的维度即为 $7 \times 7 \times 30$ 。即为最终yolov1输出结果。

结果解析- (x,y,w,h)



- 1、大红色框代表grid cell，蓝色点表示该grid cell的左上角坐标，为 $(0, 0)$ ；
- 2、深红色和绿色框代表该grid cell包含的两个 bounding box，深红点和绿色点表示这两个 bounding box 的中心坐标（ bounding box 的中心坐标一定在该 grid cell 内部）。
- 3、x, y 坐标值为归一化后的坐标值，在 0-1 之间，以左上坐标 $(0,0)$ 的为参考，因此，绿色边界框的坐标为 $(0.5,0.5)$ 。深红色边界框的坐标为 $(0.35, 0.85)$ 。

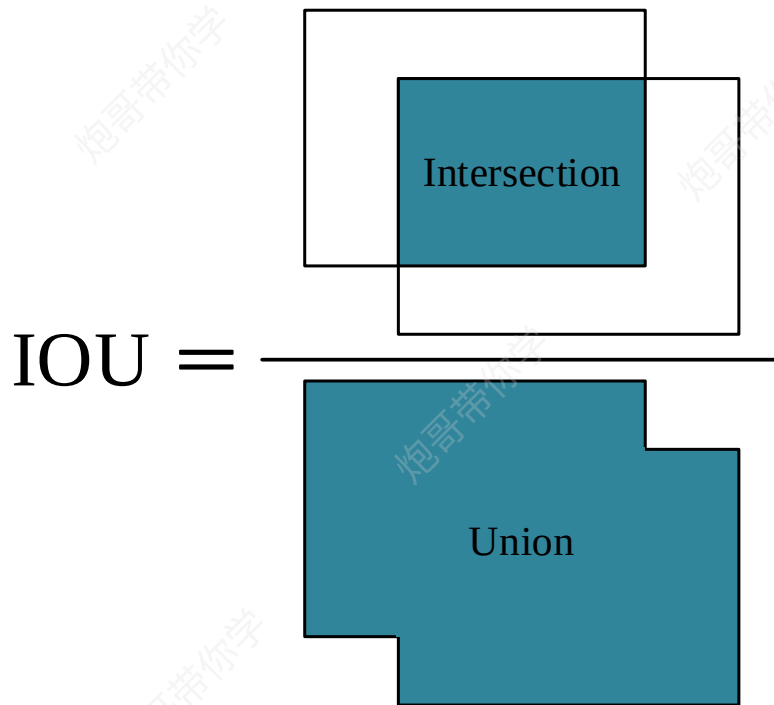


- 1、w, h: 是指该 bounding box 的宽和高，也归一化到了 0-1 之间，表示相较于原始图像的宽和高（即 448 个像素）。
- 2、比如图中的 bounding box 预测的框宽为 $w=0.43$ ，高为 $h=0.40$ 。

结果解析-置信度之IOU

IOU的全称为交并比（Intersection Over Union），通过这个名称我们大概可以猜到IOU的计算方法。IOU计算的是“预测的边框”和“真实的边框”的交集和并集的比值。

在目标检测任务中，IOU用于匹配预测的边界框（bounding box）和真实的边界框。通过计算两者之间的IOU，可以判断预测是否正确地检测到了目标。同时，在非极大值抑制计算中IOU作为其中的重要指标。

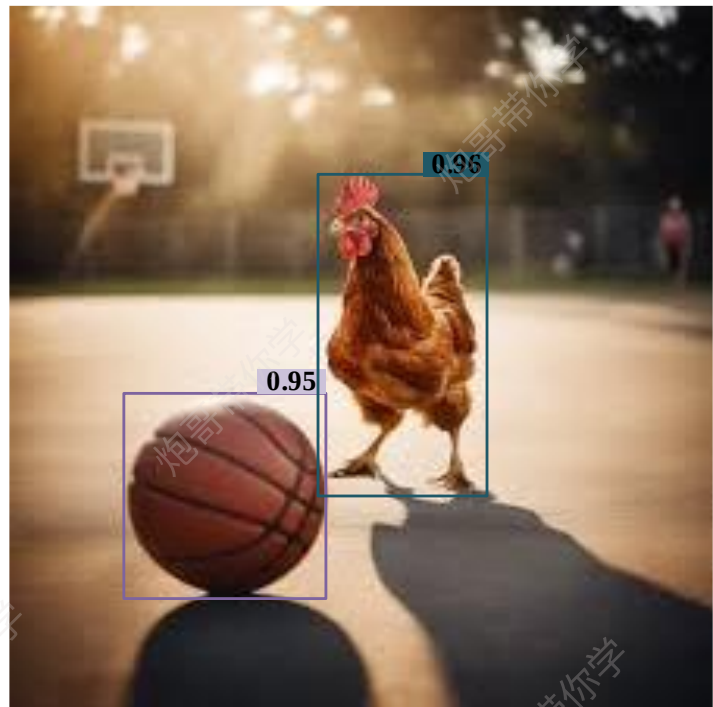


结果解析-置信度 (c)

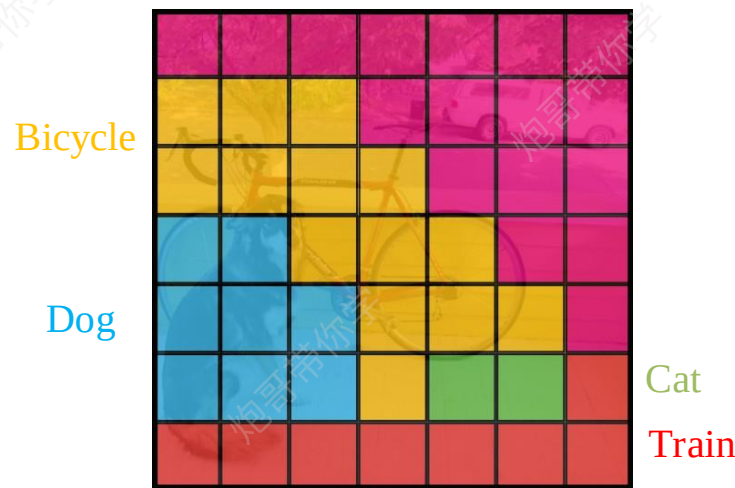
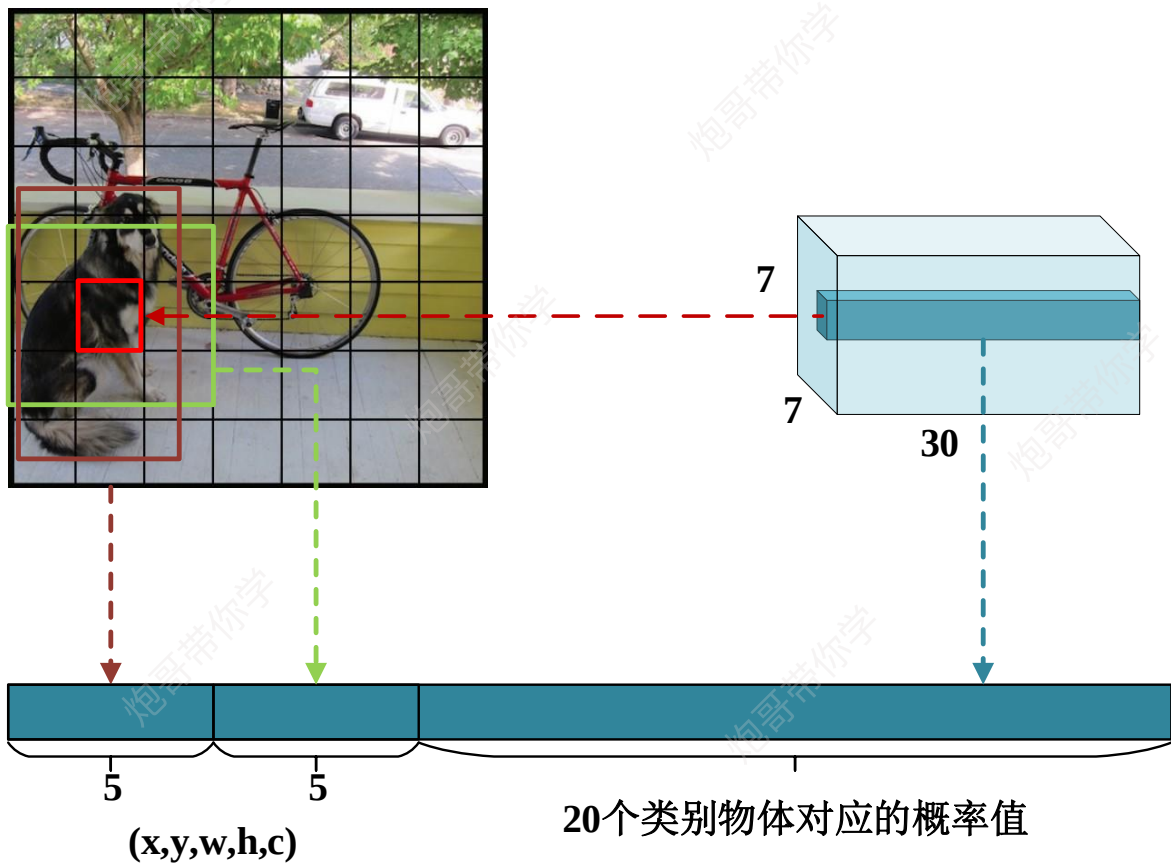
置信度作用定义有两重：

- 1、代表当前box是否有对象的概率 $\text{Pr}(\text{Object})$ ，这个概率值为1或者0，注意，这里是对象，不是某个类别的对象，也就是说它用来说明当前box内只是个背景 (background) 还是有某个物体 (对象)；
- 2、表示当前的box有对象时，它自己预测的box与物体真实的box之间的IOU交并比，**注意：**这里所说的物体真实的box实际是不存在的，这只是模型表达自己框出了物体的自信程度。因此此时置信度的公式为：

$$C = \text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

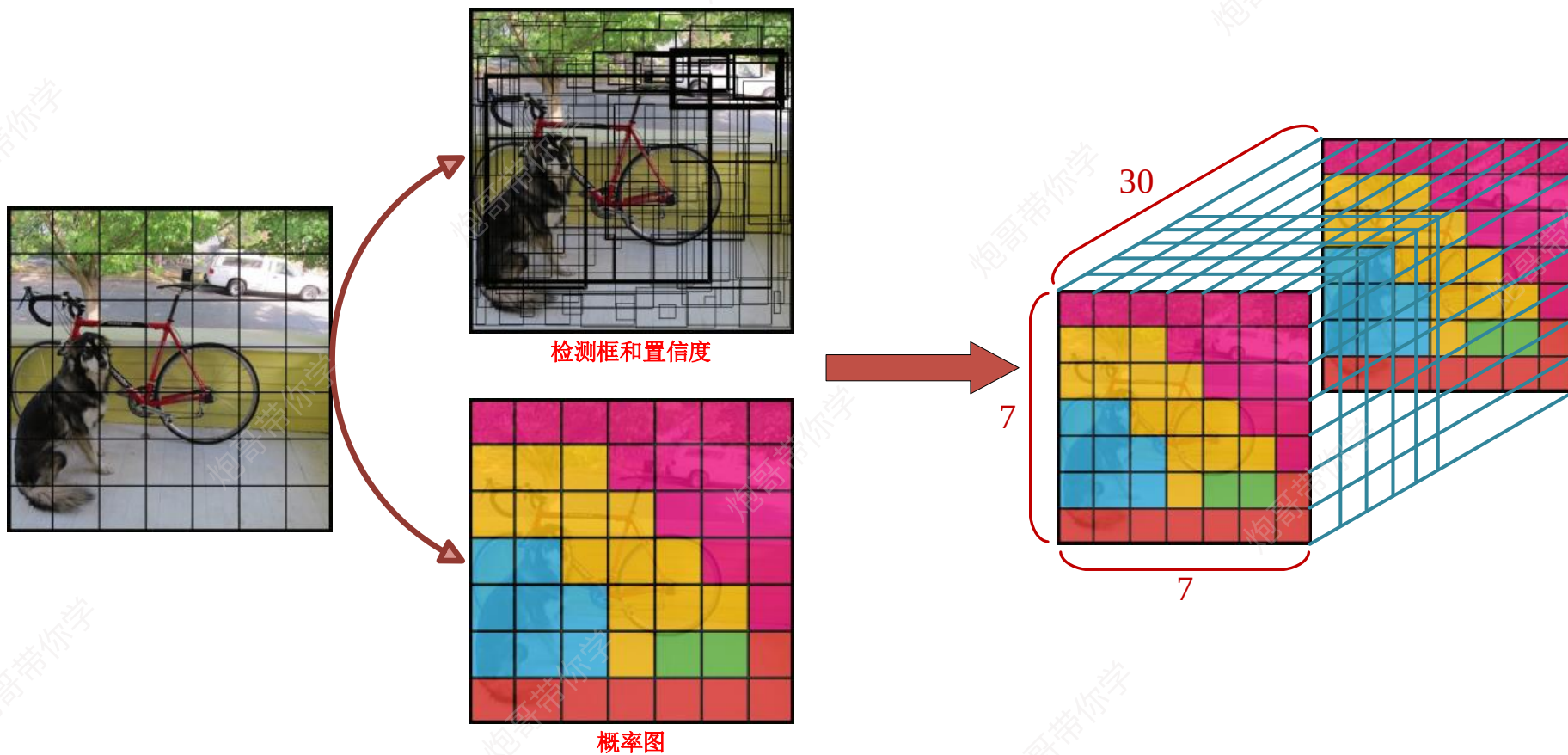


结果解析-概率值



每个小方格都会输出对应的20个类别中概率最高的类别，输出的概率利用Softmax函数进行映射，维度为 20×1

Yolov1模型输出结果解析



结果解析-类别置信度

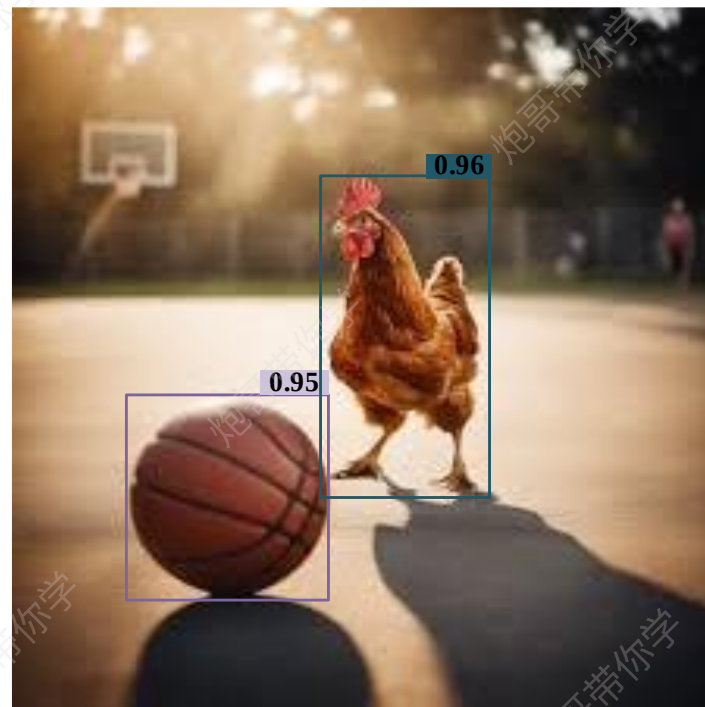
置信度的公式为：

$$C = \text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

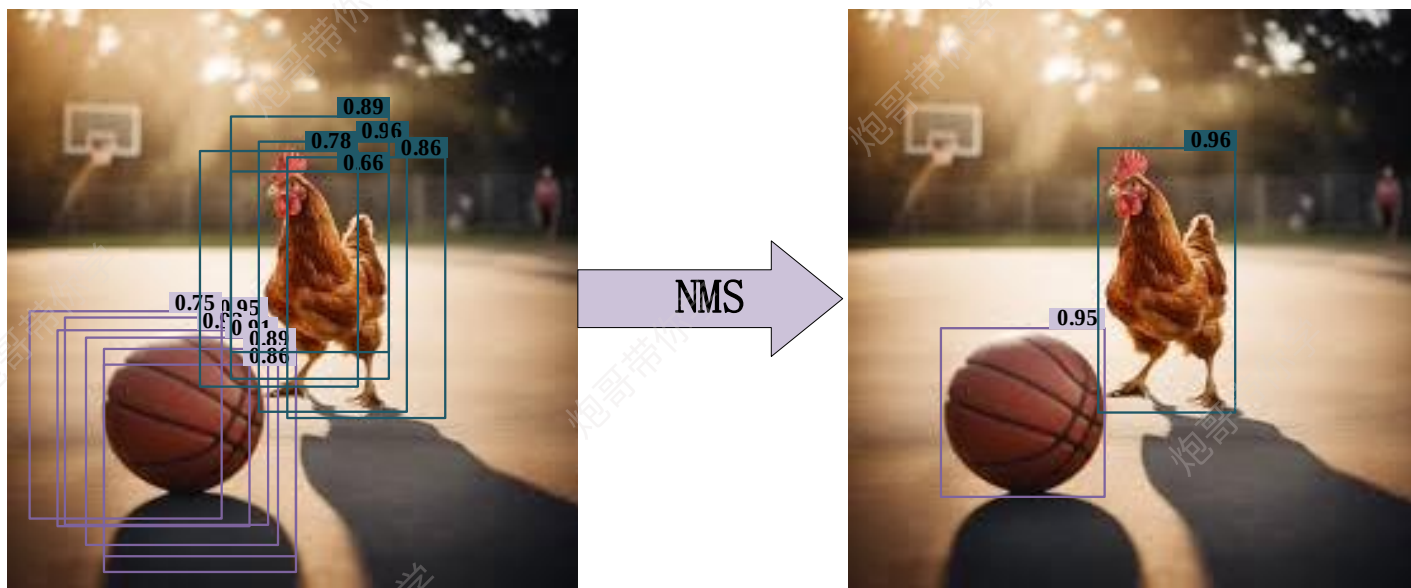
类别置信度，是在有物体的情况下，对应类别的物体的置信度，其公式为：

$$C = \text{Pr}(\text{Class}_i | \text{Object}) * \text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \text{Pr}(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

以上所述，也就不难理解作者为什么将其称之为置信度了，因为不管哪重含义，都表示一种自信程度：框出的box内确实有物体和物体类型的自信程度和框出的box将整个物体的所有特征都包括进来的自信程度。



非极大值抑制 (NMS)

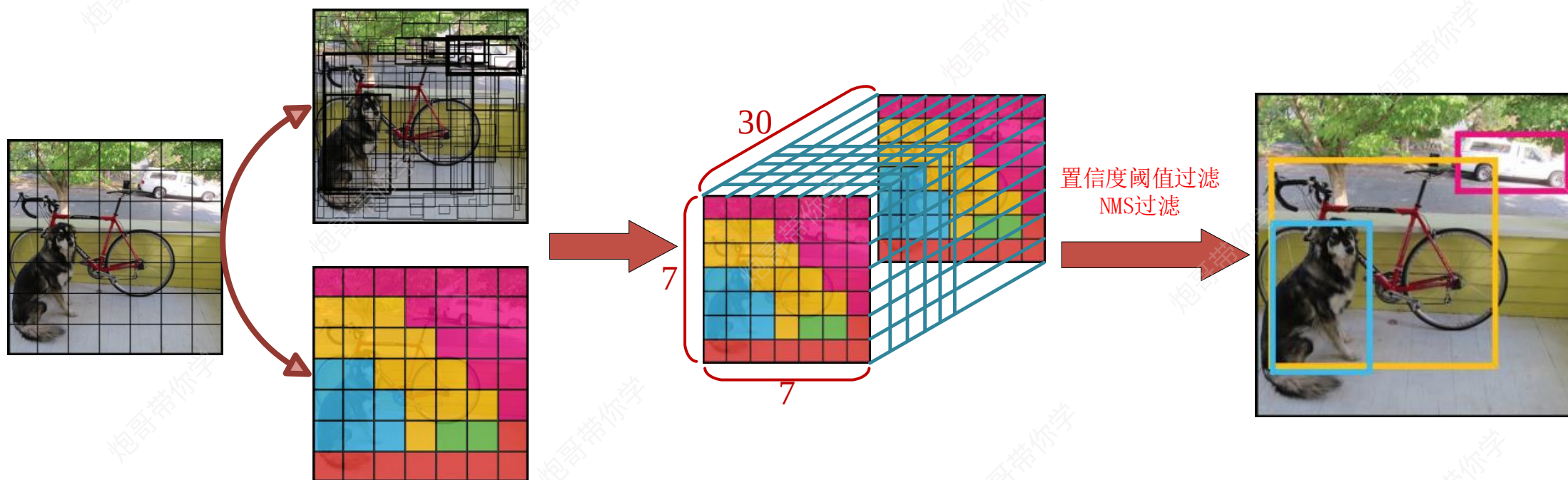


如图所示，利用Yolo系列的目标检测算法对图像中的鸡和篮球进行检测，如果不经NMS算法对其进行矩形框去重，可以发现一个目标会有多个矩形框。这是由于Yolo系列算法会针对输出图片的规定区域输出多个预测框。

NMS是针对不同类别检测框独立且并行处理，以篮球检测为例，NMS算法流程：

- 1、选取篮球检测box中的置信度最大的那一个box，记为box_best,并保留它。
- 2、计算box_best与其余篮球检测的box的IOU。在计算该步骤之前，需要设置一个IOU的阈值，该阈值往往设为0.5。
- 3、如果 $IOU > 0.5$ ，那么就舍弃这个box（由于可能两个box表示同一目标，所以保留分数高的那一个）。
- 4、从最后剩余的boxes中，再找到最大置信度的那一个，如此循环往复。

Yolov1经过后处理输出结果



Yolov1损失函数

Yolov1损失函数公式:

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

坐标损失

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

置信度损失

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

类别损失

其中:

$\sum_{i=0}^{S^2}$ 为遍历所有grid cell进行累加

$\sum_{j=0}^B$ 为遍历所有bounding box进行累加

其中:

$\sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}}$ 真实标注框中当第i个grid cell中第j个bounding box有对象的时候 $\mathbb{1}_{ij}^{\text{obj}}$ 为1, 否则为0。

$\sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}}$ 和上述相反, 无对象的时候为1, 有对象为0。

$\sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}}$ 真实标注框中当第i个grid cell中有对象的时候 $\mathbb{1}_i^{\text{obj}}$ 为1, 否则为0。

Yolov1损失函数--坐标损失

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

其中：

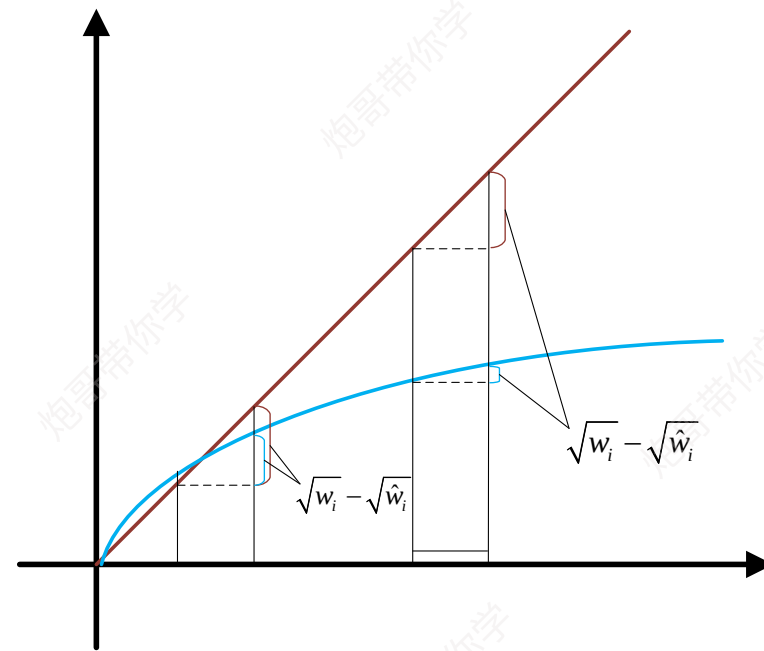
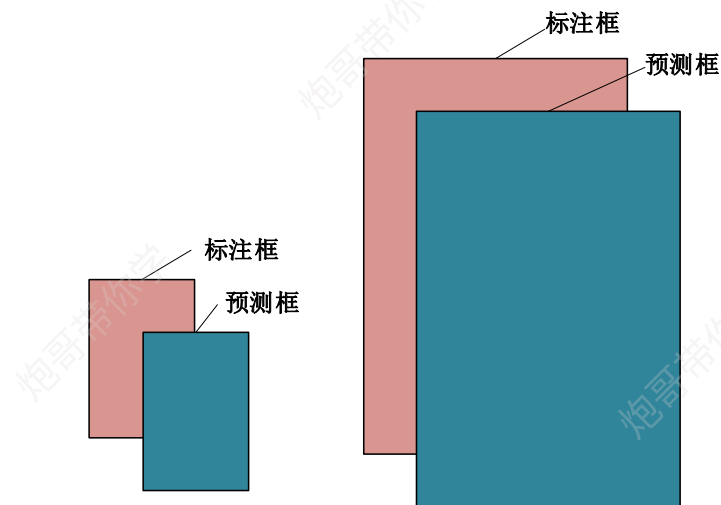
坐标损失中包含了两个损失值，一个是中心点坐标损失，一个是bounding box损失。

中心点坐标损失采用的是平方差。

bounding box损失采用的是对w，h进行开根号后进行计算平方差，具体原因如右图所示：

其中：

λ_{coord} 为5， λ_{noobj} 为0.5。数值相差较大的主要原因是，一张图片中只有少数几个目标，而绝大部分区域是没有目标的，因此加这样的系数避免loss值过渡学习到没有目标的区域。



Yolov1损失函数—置信度与类别损失

Yolov1置信度损失:

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

置信度损失针对所有的bounding box (有对象和没有对象都参与计算)。

1、当有对象的时候 $\hat{C}_i$ 标签值为**预测框和标注框之间的IOU交并比**。

2、当无对象的时候 $\hat{C}_i$ 标签值为0。

Yolov1概率损失:

$$\sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

概率损失针对所有grid cell只有有对象的grid cell 参与计算。

Yolov1训练过程

YOLOV1训练分为两个阶段：

第一阶段：

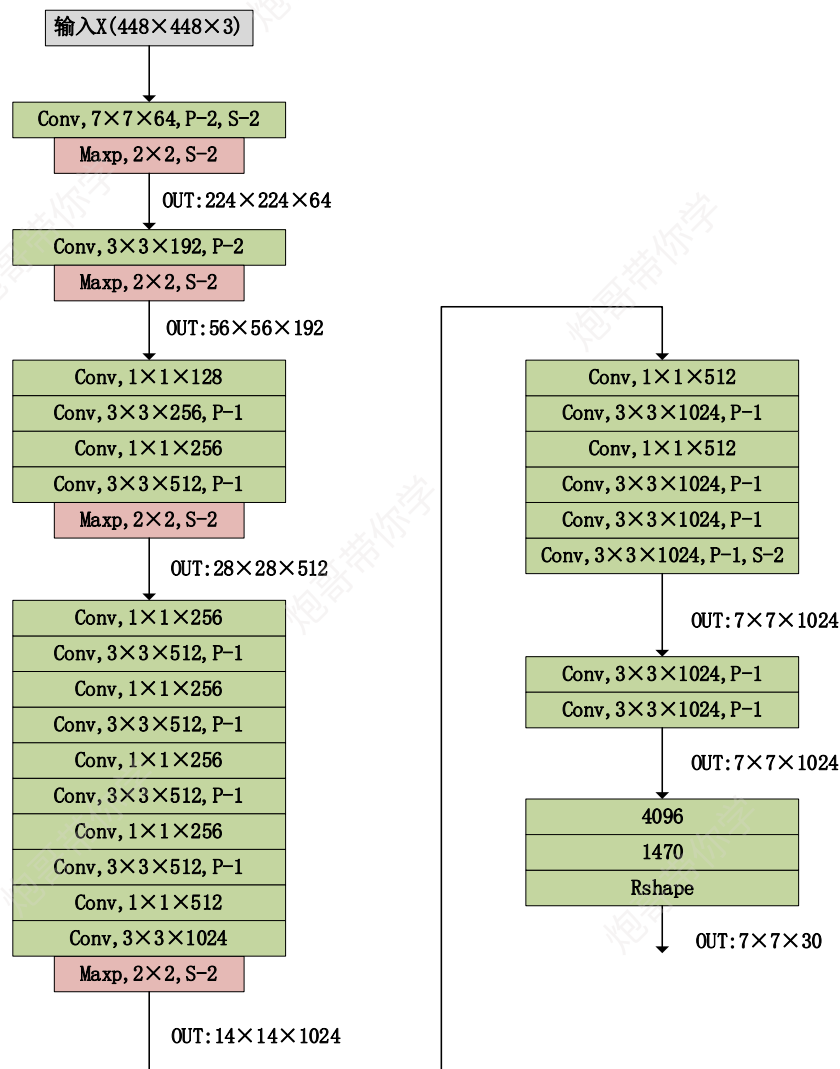
在ImageNet分类数据集上预训练分类模型。首先将yolov1的模型转化为分类模型；利用前20卷积作为模型的特征提取网络。随后利用全局平均池化层进行池化操作，随后利用全连接层进行1000分类。

此时模型的输入为224x224，对该网络进行了大约一周的训练，并在ImageNet2012验证集上实现了单个作物前5名的准确率达到88%，与传统的图像识别方法相当。

第二阶段：

将网络的输入调整为448x448。并将模型调整为检测模型，那么此时检测模型的参数如右图所示，及去除全局平均池化加入4个卷积层和两个全连接层，最后将模型的输出调整为7x7x30这样的向量大小。

在分类模型参数上继续finetune网络（135轮）



Yolov1模型总结

优势:

- 1.YOLO非常快，因为将物体检测定义为回归问题，所以检测也不需要复杂的组件。
- 2.YOLO基于全图进行检测，所以不像滑动窗口和预选区技术，YOLO中隐含着隐式编码的上下文信息。

缺点:

- 1.YOLO准确率不够高
- 2.YOLO在定位物体，尤其是小物体上表现差
- 3.可以检测到的目标物体较少

